

JAVASCRIPT FRONTEND INTERVIEW GUIDE

Basic to Intermediate | ES6+ | Logical Coding | Real-World Frontend

Tamil + English Learning Notes

Frontend JavaScript Interview Roadmap



Learn in this order: concept → syntax → code → interview question → task → revision

How to use this guide: Daily ஒரு section மட்டும் படிங்க. Concept புரிஞ்சுதும் code type பண்ணுங்க; copy-paste மட்டும் செய்யாதீங்க. ஒவ்வொரு section முடிவிலும் Interview Check + Task complete பண்ணி basic-to-intermediate frontend JS interviewsக்கு strong base கிடைக்கும்.

Prepared as a focused study guide - only high-value frontend JavaScript interview topics.

Study Order - First Read This

Interview preparationக்கு எல்லா topic-யும் random order-ல படிக்க வேண்டாம். கீழே இருக்கும் order தான் dependency-based. ஒரு topic புரியாம அடுத்ததை memorise பண்ணினா interview follow-up question-ல் stuck ஆக வாய்ப்பு அதிகம்.

1. JavaScript Core: values, types, variables, operators, coercion
2. Conditions & loops: logic building fundamentals
3. Functions: declaration, expression, arrow, parameters, return
4. Scope, hoisting, TDZ, closure
5. Arrays & objects + must-know array methods
6. ES6+: template literals, destructuring, spread/rest, modules, optional chaining
7. this, call/apply/bind, prototype & class basics
8. DOM, events, bubbling, delegation, forms
9. Browser storage: localStorage/sessionStorage/cookies concept
10. Async JavaScript: callback, Promise, async/await, event loop
11. Fetch API & HTTP handling
12. Logical coding - separate practice track
13. Real-world frontend scenarios
14. Rapid interview Q&A + final revision plan

Interview Rule: Definition மட்டும் போதாது. ஒவ்வொரு answer-லும் 4 parts target பண்ணுங்கள்: What it is -> Why we use it -> Small code/example -> One limitation or difference.

1. JavaScript Core - Must Be Clear

1.1 What is JavaScript?

JavaScript ஒரு high-level, dynamically typed programming language. Browser-ல் user interaction, DOM updates, validation, API calls போன்ற frontend behavior-களை handle பண்ண இது முக்கியமாக use ஆகிறது. Node.js மூலம் browser வெளியிலும் JavaScript run செய்யலாம்.

Interview-ல் “single-threaded” என்று கேட்டால்: JavaScript execution ஒரு call stack-ஐ use பண்ணி ஒரே நேரத்தில் ஒரு synchronous task-ஐ execute செய்கிறது. ஆனால் browser Web APIs + event loop காரணமாக timers, network calls போன்ற asynchronous work-ஐ non-blocking style-ல் handle செய்ய முடியும்.

Interview Point: JavaScript = language. ECMAScript = language specification. ES6 / ES2015 என்பது JavaScript language-க்கு பெரிய standard update.

1.2 var, let, const

`var` function-scoped; redeclare மற்றும் reassign செய்யலாம். `let` block-scoped; reassign செய்யலாம் ஆனால் same scope-ல் redeclare முடியாது. `const` block-scoped; variable binding-ஐ reassign செய்ய முடியாது. Modern JavaScript-ல் default-ஆ `const`, value change ஆக வேண்டுமென்றால் `let` use செய்வது clean practice.

```
var oldValue = 10;
let score = 20;
const appName = "Shop";

score = 30; // allowed
// appName = "New"; // TypeError
```

Common Trap: `const` object immutable என்று அர்த்தம் இல்லை. Object reference reassign முடியாது; அதன் properties update செய்யலாம்.

```
const user = { name: "Arun" };
user.name = "Bala"; // allowed
```

1.3 Data Types, typeof & null

Primitive types: string, number, bigint, boolean, undefined, symbol, null. Objects, arrays, functions ஆகியவை non-primitive/reference values. Interview-ல் primitive copy value-ஆவும் objects reference behavior-ஊம் explain செய்ய தெரிந்திருக்க வேண்டும்.

```
typeof "JS";      // "string"
typeof 10;        // "number"
typeof undefined; // "undefined"
typeof null;      // "object" <- historical JavaScript quirk
```

```
typeof [];           // "object"
typeof function(){}; // "function"
```

Interview Point: Array checkக்கு `Array.isArray(value)` use செய்யுங்கள்; `typeof []` object return பண்ணும்.

1.4 == vs === and Type Coercion

`==` comparison செய்யும் போது type conversion/coercion செய்யலாம். `===` type + value இரண்டையும் strict-ஆ compare செய்கிறது. Interview code-ல் predictable behaviorக்கு `===` recommended.

```
5 == "5";    // true
5 === "5";   // false

Boolean(""); // false
Boolean("0"); // true
Boolean(0);   // false
Boolean([]);  // true
```

Remember Falsy: `false`, `0`, `-0`, `0n`, `""`, `null`, `undefined`, `NaN` are falsy. Most other values are truthy.

1.5 Operators - Interview Essentials

- Arithmetic: `+` `-` `*` `/` `%` `**`
- Comparison: `>` `<` `>=` `<=` `===` `!==`
- Logical: `&&` `||` `!`
- Nullish coalescing: `??`
- Ternary: `condition ? trueValue : falseValue`

```
const role = "admin";
const message = role === "admin" ? "Full Access" : "Limited Access";

const name = userName ?? "Guest"; // only null/undefined fallback
```

Difference: `||` falsy values-க்கும் fallback கொடுக்கும்; `??` null அல்லது undefined மட்டும் fallback கொடுக்கும். `0` / `""` valid values என்றால் `??` useful.

Interview Check

- Why prefer const over let?
- Why does typeof null return object?
- Difference between `==` and `===`?
- Difference between `||` and `??`
- Primitive vs reference value?

Mini Task

User input age, role, subscription status வைத்து ternary + logical operators use பண்ணி access message generate செய்யுங்கள். At least 3 conditions test பண்ணுங்கள்.

2. Conditions & Loops - Logic Foundation

2.1 if / else if / switch

`if/else` ranges, complex expressions, multiple conditionsக்கு flexible. `switch` ஒரு expression-ஐ பல exact case values-க்கு compare செய்யும்போது readable. Interview-ல் “switch is replacement for all if statements” என்று சொல்லக் கூடாது.

```
const score = 82;
if (score >= 90) console.log("A");
else if (score >= 75) console.log("B");
else console.log("C");

const day = 2;
switch (day) {
  case 1: console.log("Mon"); break;
  case 2: console.log("Tue"); break;
  default: console.log("Unknown");
}
```

2.2 for / while / do...while

Iteration count roughly தெரிந்தால் `for` common. Condition true இருக்கும் வரை run பண்ண `while`. Body குறைந்தது ஒரு முறை run ஆக வேண்டுமென்றால் `do...while`. Interview logic questionsக்கு loops strong ஆக வேண்டும்; ஆனால் production code-ல் array methods பல இடங்களில் readability improve பண்ணும்.

```
for (let i = 1; i <= 5; i++) {
  console.log(i);
}

let i = 1;
while (i <= 5) {
  console.log(i);
  i++;
}
```

Complexity Starter: Single loop n elements -> $O(n)$. Nested full loop n x n -> $O(n^2)$. Interview-ல் exact Big-O deep இல்லாவிட்டாலும் basic explanation useful.

Mini Tasks

- 1 to 100 வரை even numbers sum calculate செய்யுங்கள்.
- ஒரு number prime ஆ check செய்ய loop use செய்யுங்கள்.
- 1-5 multiplication table nested loop மூலம் print செய்யுங்கள்.

3. Functions - Very High Interview Priority

3.1 Declaration, Expression, Arrow

Function declaration named function syntax; declaration fully hoisted ஆகும். Function expression ஒரு variable-ல் function value store செய்கிறது. Arrow function ES6 concise syntax; lexical `this` காரணமாக normal function-க்கு 100% replacement இல்லை.

```
function add(a, b) {  
  return a + b;  
}  
  
const subtract = function (a, b) {  
  return a - b;  
};  
  
const multiply = (a, b) => a * b;
```

Interview Point: Arrow functionக்கு own `this`, `arguments`, `prototype` இல்லை. Constructor-ஆ `new` உடன் use செய்ய முடியாது.

3.2 Parameters, Arguments, Default, Rest

```
const total = (tax = 0, ...prices) => {  
  return prices.reduce((sum, price) => sum + price, 0) + tax;  
};  
  
console.log(total(10, 100, 200)); // 310
```

Parameter என்பது function definition-ல் இருக்கும் placeholder. Argument என்பது function call-ல் pass செய்யப்படும் actual value. Rest parameter `...prices` பல arguments-ஐ ஒரு array-ஆ collect செய்கிறது.

3.3 Higher-Order Functions & Callbacks

ஒரு function மற்றொரு function-ஐ argument-ஆ accept செய்தாலோ அல்லது function return செய்தாலோ higher-order function என்று சொல்லலாம். Callback என்பது இன்னொரு functionக்கு pass செய்யப்பட்டு பின்னர் execute ஆகும் function. Array methods மற்றும் async programming இரண்டிலும் இது frequent interview topic.

```
const process = (name, callback) => {  
  const result = name.trim();  
  callback(result);  
};  
  
process(" Arun ", value => console.log(value));
```

3.4 Pure Function - Useful Frontend Concept

Same inputக்கு same output return செய்து external state-ஐ mutate செய்யாத function pure function. React state logic, reducers, testing போன்ற இடங்களில் இது மிகவும் useful mental model.

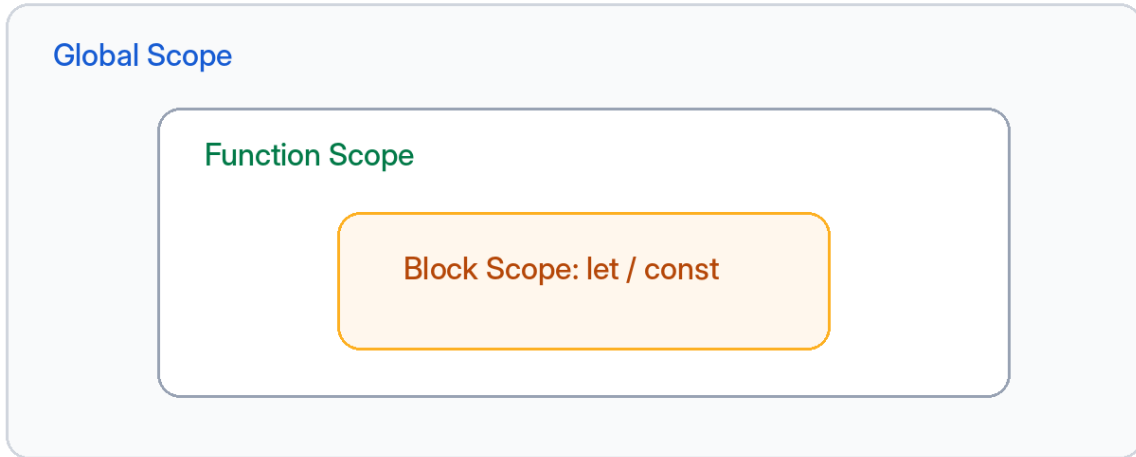
```
const addTax = (price, tax) => price + tax; // pure
```

Interview Check

- Function declaration vs expression hoisting?
- Arrow vs normal function?
- Parameter vs argument?
- What is callback?
- What makes a function higher-order?

4. Scope, Hoisting, TDZ & Closure

Scope + Hoisting + Closure Mental Model



Closure = inner function remembers variables from its lexical outer scope.

4.1 Scope & Lexical Scope

Scope என்பது variable எங்கே accessible என்று தீர்மானிக்கும் rule. JavaScript lexical scope use செய்கிறது: function எங்கே எழுதப்பட்டதோ அந்த surrounding code structure அடிப்படையில் outer variables access ஆகும்; function எங்கே call ஆனது அடிப்படையில் இல்லை.

```
const app = "Store";

function outer() {
  const user = "Arun";
  function inner() {
    console.log(app, user);
  }
  inner();
}
outer();
```

4.2 Hoisting & TDZ

Executionக்கு முன் declarations environment-ல் register ஆகும் behavior-ஐ hoisting என்று explain செய்யலாம். Function declarations callable before declaration. `var` declaration `undefined`-ஆ initialized. `let`/`const` declaration scope start-லிருந்து declaration line வரை Temporal Dead Zone-ல் இருக்கும்; access செய்தால் ReferenceError.

```
console.log(a); // undefined
var a = 10;
```

```
// console.log(b); // ReferenceError
let b = 20;
```

4.3 Closure

Outer function complete ஆன பிறகும் inner function தனது lexical outer scope variables-ஐ access/remember செய்யும் behavior closure. Private state, factories, event handlers, memoization போன்ற patterns-ல் closures use ஆகும்.

```
function createCounter() {
  let count = 0;
  return () => ++count;
}

const counter = createCounter();
console.log(counter()); // 1
console.log(counter()); // 2
```

Real World: Search input debounce, button click counters, factory functions, module/private state—all closure idea use செய்யலாம்.

Task

`createWallet(initialBalance)` function எழுதுங்கள். Returned object-ல் deposit(), withdraw(), getBalance() methods இருக்கட்டும்; balance direct access ஆகக்கூடாது. Closure use செய்யுங்கள்.

5. Arrays & Objects - Frontend Daily Work

5.1 Array Core Methods

Frontend data பெரும்பாலும் API-லிருந்து array of objects ஆக வரும். Interviewக்கு mutation செய்கிற methods vs new array return செய்கிற methods தெரிய வேண்டும். `push/pop/shift/unshift/splice/sort` mutate செய்யலாம்; `map/filter/slice/concat/toSorted` போன்றவை original array-ஐ mutate செய்யாமல் புதிய value return செய்யும் patternsக்கு use ஆகும்.

Method	Use	Returns	Mutates?
forEach	side effect / iterate	undefined	No*
map	transform every item	new array	No
filter	keep matching items	new array	No
find	first matching item	item / undefined	No
some	any match?	boolean	No
every	all match?	boolean	No
reduce	combine to one value	any accumulator	No
push	add at end	new length	Yes
splice	add/remove in place	removed array	Yes
slice	copy portion	new array	No

*forEach itself does not change the array structure, but your callback can mutate objects/variables.

```
const users = [
  { id: 1, name: "Arun", active: true },
  { id: 2, name: "Bala", active: false }
];

const names = users.map(user => user.name);
const activeUsers = users.filter(user => user.active);
const user = users.find(user => user.id === 2);
const hasActive = users.some(user => user.active);
```

5.2 Objects, Destructuring & Spread

```
const user = { name: "Arun", age: 22, city: "Madurai" };
const { name, city } = user;

const updatedUser = { ...user, city: "Chennai" };
const clone = { ...user }; // shallow copy
```

Spread syntax shallow copy மட்டுமே. Nested object references shared ஆகலாம். Interview-ல் shallow vs deep copy difference கேட்கப்படும். Modern environments-ல் supported dataக்கு `structuredClone()` deep clone option.

```
const original = { profile: { city: "Madurai" } };
const copy = { ...original };
copy.profile.city = "Chennai";
console.log(original.profile.city); // Chennai
```

Interview Point: Array/object comparison reference-based: `{ } === { }` false, `[ ] === [ ]` false. Same reference மட்டும் true.

5.3 reduce - Minimum Interview Understanding

```
const cart = [100, 200, 50];  
const total = cart.reduce((sum, price) => sum + price, 0);  
console.log(total); // 350
```

`reduce(callback, initialValue)` array items-ஐ ஒரு accumulated result-ஆ combine செய்கிறது. Sum, grouping, frequency count போன்ற questions-ல் useful; ஆனால் simple map/filter use case-ஐ unnecessarily reduce பண்ண வேண்டாம்.

Tasks

- Products array-லிருந்து price > 1000 products filter செய்யுங்கள்.
- Employees array-ல் id வைத்து employee find செய்யுங்கள்.
- Cart array total reduce மூலம் calculate செய்யுங்கள்.
- Users array-ஐ name மட்டும் கொண்ட array-ஆ map செய்யுங்கள்.

6. ES6+ - Interview Essentials Only

ES6+ என்பது modern JavaScript syntax/features-க்கு பொதுவாக பயன்படுத்தப்படும் term. Frontend interviews-ல் syntax தெரிந்ததுடன், எந்த problem-ஐ simplify செய்கிறது என்று explain செய்ய வேண்டும்.

6.1 Template Literals

```
const name = "Arun";
const msg = `Welcome ${name}`;
```

6.2 Destructuring

```
const [first, second] = [10, 20];
const { id, email } = user;
```

6.3 Spread vs Rest

```
const copy = [...numbers]; // spread
const user2 = { ...user, age: 23 };

const sum = (...values) => values.reduce((a,b) => a+b, 0); // rest
```

Difference: Same `...` syntax; context decides meaning. Spread = expand. Rest = collect.

6.4 Optional Chaining & Nullish Coalescing

```
const city = user?.address?.city ?? "Unknown";
```

6.5 Modules

```
// utils.js
export const add = (a, b) => a + b;
export default function greet() { return "Hi"; }

// app.js
import greet, { add } from "./utils.js";
```

6.6 Short Interview List

- let / const
- arrow functions
- template literals

- default parameters
- destructuring
- spread / rest
- modules
- classes
- promises
- optional chaining
- nullish coalescing

Task

ஒரு nested API user object create பண்ணி destructuring, spread update, optional chaining, nullish coalescing அனைத்தையும் ஒரே small program-ல் use செய்யுங்கள்.

7. this, call/apply/bind, Prototype & Class Basics

7.1 `this` - Interview-Level Understanding

`this` value function எப்படி call செய்யப்படுகிறது என்பதின் அடிப்படையில் மாறும் (arrow functions exception: lexical this). Object method-ல் method call object usually `this`; standalone regular function strict mode-ல் `undefined`; constructor call-ல் new object; arrow function surrounding `this` capture செய்கிறது.

```
const user = {
  name: "Arun",
  show() { console.log(this.name); }
};
user.show(); // Arun

const arrow = () => console.log(this); // lexical this
```

7.2 call, apply, bind

```
function greet(city) {
  return `${this.name} from ${city}`;
}
const person = { name: "Arun" };

greet.call(person, "Madurai");
greet.apply(person, ["Madurai"]);
const bound = greet.bind(person, "Madurai");
bound();
```

`call` immediately execute + arguments separately. `apply` immediately execute + arguments array. `bind` new bound function return செய்கிறது; later execute செய்யலாம்.

7.3 Prototype & Class

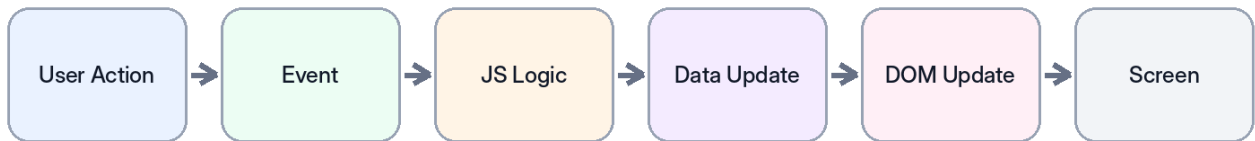
JavaScript inheritance prototype-based. Objects மற்ற object/prototype-லிருந்து properties/methods lookup செய்யலாம். ES6 `class` syntax prototype systemக்கு cleaner syntax layer; JavaScript class-based language ஆக மாறவில்லை.

```
class User {
  constructor(name) { this.name = name; }
  greet() { return `Hi ${this.name}`; }
}
const user1 = new User("Arun");
```

Interview Focus: Deep prototype chain internals beginner interviewக்கு அவசியம் இல்லை; `prototype`, inheritance concept, class syntax, `this` basics clear இருந்தால் போதும்.

8. DOM & Events - Frontend Interview Core

DOM Interaction Flow



Example: click → addEventListener → validate → array.push → render() → innerHTML

Interview idea: Event delegation uses bubbling so one parent listener can handle many child elements.

8.1 DOM Selection & Update

DOM browser உருவாக்கும் document tree. JavaScript மூலம் element select, content/style/attributes update, elements create/remove செய்யலாம். Frontend interview-ல் direct DOM manipulation syntax மற்றும் event flow clear இருக்க வேண்டும்.

```
const title = document.querySelector("#title");
title.textContent = "Updated";
title.classList.toggle("active");

const item = document.createElement("li");
item.textContent = "New Item";
document.querySelector("ul").append(item);
```

8.2 Events, Bubbling & Delegation

Event bubbling-ல் child element event parent ancestors நோக்கி bubble ஆகும். Event delegation இந்த behavior-ஐ use பண்ணி parent-ல் ஒரு listener வைத்து dynamically added child items உட்பட பல children-ஐ handle செய்யும். Large listsக்கு இது useful.

```
document.querySelector("#list").addEventListener("click", (event) => {
  if (event.target.matches("button.delete")) {
    event.target.closest("li").remove();
  }
});
```

Must Know: `event.target` = event originated element. `event.currentTarget` = listener attached element. `preventDefault()` default browser action stop. `stopPropagation()` propagation stop.

8.3 Form Flow

```
form.addEventListener("submit", (event) => {  
  event.preventDefault();  
  const name = nameInput.value.trim();  
  if (!name) return;  
  const user = { name };  
  users.push(user);  
  renderUsers();  
});
```

Interview-ல் input -> validate -> object -> array/state -> render என்ற data flow explain பண்ணுவது practical frontend understanding கட்டும்.

Task

Todo list build செய்யுங்கள்: form submit, array of objects, render function, delete button event delegation, completed class toggle. No framework.

9. Browser Storage - Practical Interview Notes

9.1 localStorage & sessionStorage

Web Storage values strings. Objects/arrays save செய்ய `JSON.stringify()`, read செய்ய `JSON.parse()` use செய்கிறோம். `localStorage` browser sessions கடந்து persist ஆகும்; `sessionStorage` tab/session lifetimeக்கு scoped.

```
const users = [{ id: 1, name: "Arun" }];
localStorage.setItem("users", JSON.stringify(users));

const saved = JSON.parse(localStorage.getItem("users")) || [];
```

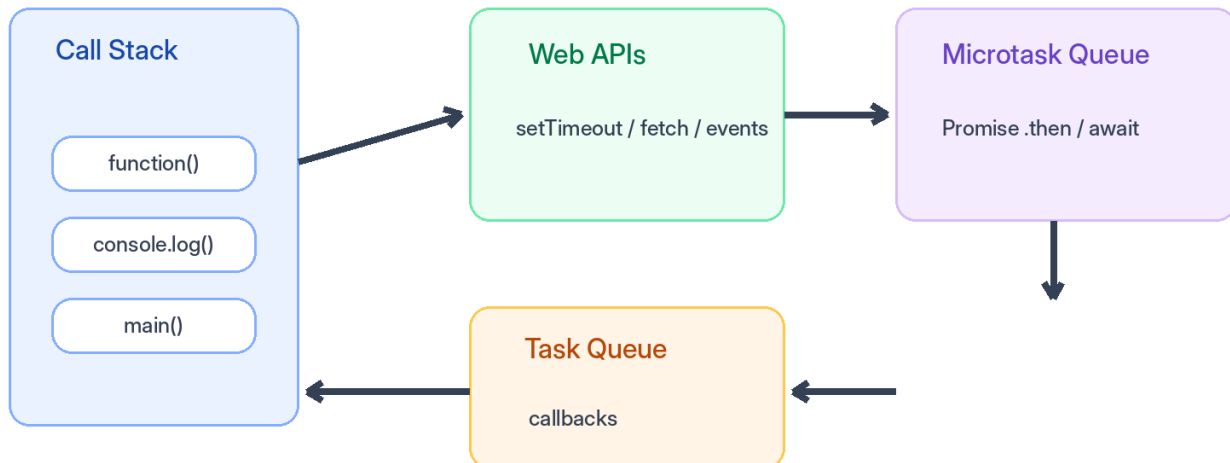
Security: Passwords, access tokens போன்ற sensitive data-ஐ blindly localStorage-ல் store செய்வது XSS risk காரணமாக production auth designக்கு safe default இல்லை. HttpOnly secure cookies போன்ற approaches backend/security requirements அடிப்படையில் use செய்யப்படும்.

9.2 localStorage vs sessionStorage vs Cookie

Feature	localStorage	sessionStorage	Cookie
Lifetime	Until removed	Tab/session	Can expire
Sent to server automatically	No	No	Yes, depending on cookie
Typical use	Preferences/cache	Temporary tab data	Session/auth scenarios
API	Web Storage	Web Storage	document.cookie / server headers

10. Async JavaScript, Promise & Event Loop

JavaScript Event Loop - Interview View



Rule: Call stack must be empty. Microtasks run before normal task-queue callbacks.

10.1 Callback -> Promise -> async/await

Callbacks async completion handle செய்ய பழைய/common pattern. Nested callbacks readability குறைக்கலாம். Promise future async result-ஐ pending/fulfilled/rejected states-ல் represent செய்கிறது. `async/await` Promise-ன் மேல் cleaner syntax; underlying async model Promiseதான்.

```
const getData = () =>
  fetch("/api/users")
    .then(res => {
      if (!res.ok) throw new Error("Request failed");
      return res.json();
    });

async function load() {
  try {
    const data = await getData();
    console.log(data);
  } catch (error) {
    console.error(error.message);
  }
}
```

10.2 Event Loop - The Interview Explanation

Synchronous code call stack-ல் run ஆகும். Browser timers/network/event handling போன்ற work-ஐ Web APIs handle செய்கின்றன. Completion callbacks queues-ல் wait செய்கின்றன; call stack empty ஆனதும் event loop eligible callbacks-ஐ stackக்கு schedule செய்கிறது.

Promise reactions (`then/catch/finally`) மற்றும் `await` continuation microtask queue-ல் scheduled. Timers போன்ற callbacks task/macrotask queue-ல். ஒரு task முடிந்த பிறகு microtasks drain ஆகும் என்பதால் Promise callbacks பொதுவாக `setTimeout(..., 0)` callbackக்கு முன் run ஆகும்.

```
console.log("A");
setTimeout(() => console.log("B"), 0);
Promise.resolve().then(() => console.log("C"));
console.log("D");

// A, D, C, B
```

Classic Interview: Output order explain செய்ய call stack -> microtask queue -> task queue sequence சொல்லுங்கள்.

10.3 Promise Combinators - Know the Difference

- `Promise.all()`: all fulfill; one reject -> whole reject
- `Promise.allSettled()`: wait for all outcomes
- `Promise.race()`: first settled result
- `Promise.any()`: first fulfilled result; all reject -> `AggregateError`

Task

மேலுள்ள A/B/C/D example மாதிரி 5 output prediction snippets எழுதுங்கள். ஒவ்வொன்றுக்கும் “sync / microtask / task” என்று label பண்ணி explain செய்யுங்கள்.

11. Fetch API & HTTP - Frontend Real-World Flow

Async API Flow



Success: response.ok → parse → use data | Failure: throw / catch → show fallback

11.1 GET with Error Handling

```
async function getUsers() {
  try {
    const response = await fetch("https://jsonplaceholder.typicode.com/users");
    if (!response.ok) throw new Error(`HTTP ${response.status}`);
    const users = await response.json();
    return users;
  } catch (error) {
    console.error("Load failed:", error.message);
    return [];
  }
}
```

Important: `fetch()` network failure-ல் reject ஆகலாம்; ஆனால் HTTP 404/500 response கிடைத்ததற்காக மட்டும் automatic reject ஆகாது. அதனால் `response.ok` அல்லது status check practical code-ல் முக்கியம்.

11.2 POST

```
async function createUser(user) {
  const response = await fetch("/api/users", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(user)
  });
  if (!response.ok) throw new Error("Create failed");
  return response.json();
}
```

11.3 CRUD Memory

- GET -> read data
- POST -> create
- PUT -> replace/update full resource conventionally
- PATCH -> partial update
- DELETE -> remove

Real Frontend Flow: Loading state -> fetch -> response check -> parse -> render success. Failure என்றால் error UI. `finally`/cleanup-ல் loading false.

Task

Users API fetch பண்ணி cards display செய்யுங்கள். Loading text, error message, retry button சேர்க்கவும். Separate `fetchUsers()` and `renderUsers()` functions use பண்ணுங்கள்.

12. Logical Coding - Separate Interview Track

How to Solve a JavaScript Coding Question



Logical roundக்கு syntax மட்டும் போதாது. First brute-force/simple correct solution எழுதுங்கள்; பின்னர் readability, edge cases, complexity improve செய்யுங்கள். Interview-ல் silent-ஆ code எழுதாமல் input/output example பேசுவது நல்ல signal.

12.1 Level 1 - Must Solve Without Help

15. Reverse a string without using reverse().
16. Check palindrome string/number.
17. Find factorial of a number.
18. Generate Fibonacci series for n terms.
19. Check prime number.
20. Find largest and smallest number in array.
21. Sum of array values.
22. Count vowels in a string.
23. Count even and odd numbers in array.
24. Find second largest number without sorting entire array.

12.2 Level 2 - Common Frontend Interview Logic

25. Remove duplicate values from array.
26. Create frequency count of characters/array values.
27. Find duplicate values in an array.
28. Find missing number from 1..n.
29. Anagram check for two strings.
30. Flatten one-level nested array; then discuss recursive flatten.

31. Group array of objects by a property.
32. Sort objects by numeric field without mutating original array.
33. Find total cart amount from array of objects.
34. Convert array of objects to lookup object keyed by id.

12.3 Model Pattern 1 - Frequency Counter

```
function charFrequency(text) {
  const count = {};
  for (const ch of text.toLowerCase()) {
    if (ch === " ") continue;
    count[ch] = (count[ch] ?? 0) + 1;
  }
  return count;
}
console.log(charFrequency("hello"));
```

இந்த pattern duplicate, anagram, occurrence count போன்ற பல questionsக்கு reusable. Interview insight என்பது answer memorize பண்ணுவது இல்ல; pattern identify பண்ணுவது.

12.4 Model Pattern 2 - Two-Pointer Thinking

```
function isPalindrome(text) {
  const value = text.toLowerCase();
  let left = 0;
  let right = value.length - 1;
  while (left < right) {
    if (value[left] !== value[right]) return false;
    left++;
    right--;
  }
  return true;
}
```

12.5 Model Pattern 3 - Object Lookup

```
function usersById(users) {
  return users.reduce((lookup, user) => {
    lookup[user.id] = user;
    return lookup;
  }, {});
}
```

12.6 Complexity - Enough for Frontend Interviews

- $O(1)$: constant lookup/example property access
- $O(n)$: scan one list
- $O(n \log n)$: typical comparison sort
- $O(n^2)$: nested full comparison loops

Do Not Overcomplicate: Frontend junior/intermediate round-ல் clean correct code + explanation + edge

case handling often more valuable than clever one-liner.

13. Real-World Frontend JavaScript Scenarios

Scenario 1 - Search Filter

Requirement: user types name, matching users மட்டும் show. Interview flow: input event -> normalize query -> filter array -> render. Empty query என்றால் full list அல்லது empty state requirement clarify செய்யுங்கள்.

```
searchInput.addEventListener("input", () => {
  const q = searchInput.value.trim().toLowerCase();
  const filtered = users.filter(user =>
    user.name.toLowerCase().includes(q)
  );
  renderUsers(filtered);
});
```

Scenario 2 - Cart Total

```
const total = cart.reduce((sum, item) => {
  return sum + item.price * item.quantity;
}, 0);
```

Follow-up: discount, tax, quantity validation, currency formatting. Interview-ல் requirement expand ஆனால் logic small functions-ஆ split பண்ணுங்கள்.

Scenario 3 - Form Validation

```
function validate({ name, email, password }) {
  const errors = {};
  if (!name.trim()) errors.name = "Name required";
  if (!email.includes("@")) errors.email = "Invalid email";
  if (password.length < 8) errors.password = "Minimum 8 characters";
  return errors;
}
```

Scenario 4 - Debounce Concept

Search input ஒவ்வொரு keypressக்கும் API call செய்தால் unnecessary requests அதிகம். Debounce user typing ஒரு short delay நிறுத்திய பிறகு function execute செய்ய உதவும். Closure + timer interview concepts இரண்டும் இதில் வரும்.

```
function debounce(fn, delay) {
  let timer;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), delay);
  };
}
```

```
const onSearch = debounce(value => console.log(value), 300);
```

Scenario 5 - Immutable Update

```
const updatedUsers = users.map(user =>
  user.id === id ? { ...user, active: true } : user
);
```

இந்த pattern Reactக்கு மிகவும் important. Existing array/object mutate செய்யாமல் new structure create செய்யும் habit frontend interviews-ல் useful.

14. Rapid Interview Q&A - Say It in 20-40 Seconds

What is closure?

Inner function அதன் lexical outer scope variables-ஐ outer function execution முடிந்த பிறகும் access செய்யும் behavior.

What is hoisting?

Declarations executionக்கு முன் environment-ல் registered ஆகும் behavior; var undefined initialization, let/const TDZ, function declaration callable before declaration.

map vs forEach?

map transformed new array return செய்கிறது; forEach iteration/side-effectsக்கு, return value undefined.

filter vs find?

filter all matches new array; find first match item or undefined.

slice vs splice?

slice non-mutating copy portion; splice original array mutate செய்து add/remove செய்யும்.

Shallow vs deep copy?

Shallow copy top-level மட்டும் copy; nested references shared. Deep copy nested structures-யும் independent copy.

Promise states?

pending, fulfilled, rejected.

async function returns what?

Always a Promise; returned value fulfilled Promise-ஆ wrapped ஆகும்.

Why response.ok?

fetch HTTP 4xx/5xxக்கு automatic reject ஆகாமல் response resolve செய்யலாம்; status success check செய்ய.

Event bubbling?

Event target-லிருந்து ancestor elements நோக்கி propagation ஆகும் phase.

Event delegation?

Parent listener வைத்து bubbling மூலம் many child events handle செய்வது.

== vs ===?

== coercion செய்யலாம்; === type + value strict compare.

null vs undefined?

undefined usually missing/uninitialized; null intentional empty value.

this in arrow function?

Own this இல்லை; surrounding lexical this capture செய்கிறது.

Spread vs rest?

Spread expand; rest collect.

localStorage limitation?

String storage, synchronous API, origin-scoped; sensitive data storageக்கு security concerns.

What is event loop?

Call stack empty ஆனபோது queued async callbacks/microtasks execution scheduling coordinate செய்யும் mechanism.

Microtask example?

Promise callbacks and await continuation.

What is pure function?

Same input -> same output; external observable state mutate செய்யாது.

Why immutability?

Predictable updates, easier comparison/debugging, React state patternsக்கு helpful.

15. Interview Practice Tasks - Do, Don't Just Read

A. Core JS - 10 Tasks

- 35. Falsy values test program எழுதுங்கள்; 10 different values output predict செய்யுங்கள்.
- 36. == vs === five examples எழுதுங்கள் and reason சொல்லுங்கள்.
- 37. Ternary use பண்ணி age group label generate செய்யுங்கள்.
- 38. Nested object optional chaining example build செய்யுங்கள்.
- 39. `||` vs `??` 0 and empty string examples மூலம் prove செய்யுங்கள்.
- 40. var/let/const scope example எழுதுங்கள்.
- 41. Hoisting output prediction 5 snippets.
- 42. Closure counter + private wallet build செய்யுங்கள்.
- 43. Arrow vs regular `this` simple object example எழுதுங்கள்.
- 44. call/apply/bind same functionக்கு demonstrate செய்யுங்கள்.

B. Arrays & Objects - 10 Tasks

- 45. Users list active users filter.
- 46. Names array map.
- 47. ID user find.
- 48. Cart reduce total.
- 49. some/every eligibility check.
- 50. Duplicate remove.
- 51. Frequency object.
- 52. Sort copy without mutating original.
- 53. Nested object shallow-copy problem demonstrate.
- 54. Group employees by department.

C. DOM + Browser - 6 Tasks

- 55. Todo CRUD UI with array of objects.
- 56. Search filter UI.
- 57. Theme toggle with localStorage.
- 58. Form validation with inline errors.
- 59. Event delegation list delete.
- 60. Dynamic table from API data.

D. Async - 6 Tasks

- 61. Promise resolve/reject manually create.
- 62. A/D/C/B event-loop output explain.
- 63. Fetch GET with loading/error.
- 64. POST form data.
- 65. Promise.all two API requests.
- 66. Debounced search function.

16. 14-Day Interview Revision Plan

Day	Focus	Output
Day 1	Variables, types, equality, truthy/falsy, operators	Code + explain aloud
Day 2	Conditions, loops + 5 logic problems	Code + explain aloud
Day 3	Functions, arrow, callbacks, HOF	Code + explain aloud
Day 4	Scope, hoisting, TDZ, closure	Code + explain aloud
Day 5	Arrays methods: map/filter/find/reduce/some/every	Code + explain aloud
Day 6	Objects, destructuring, spread/rest, copy	Code + explain aloud
Day 7	ES6+ + this/call/apply/bind	Code + explain aloud
Day 8	Prototype/class basics + revision quiz	Code + explain aloud
Day 9	DOM selectors, create/update/remove	Code + explain aloud
Day 10	Events, bubbling, delegation + form task	Code + explain aloud
Day 11	Storage + real-world small app	Code + explain aloud
Day 12	Promise, async/await, event loop	Code + explain aloud
Day 13	Fetch/CRUD + API UI	Code + explain aloud
Day 14	10 logic questions + rapid Q&A mock interview	Code + explain aloud

Final Interview Method

- Question கேட்டதும் 5-10 seconds requirement clarify செய்யுங்கள்.
- Example input/output சொல்லுங்கள்.
- Simple solution first எழுதுங்கள்.
- Variables/function names readable-ஆ வைத்துக்கொள்ளுங்கள்.
- Edge cases test செய்யுங்கள்: empty, null, duplicate, zero, negative where relevant.
- Complexity basic level explain செய்யுங்கள்.
- Code முடிந்ததும் line-by-line பேசுங்கள்; interviewer guess பண்ண விடாதீங்க.

The Goal: Syntax memorization அல்ல. “Data வந்தது -> transform பண்ணினேன் -> condition/logic apply பண்ணினேன் -> UI/API result handle பண்ணினேன்” என்று end-to-end explain செய்யும் confidence build பண்ண வேண்டும்.

Final Must-Revise Checklist

- var/let/const + scope + hoisting + TDZ
- ==/=== + coercion + truthy/falsy
- functions + arrow + callback + closure
- arrays: map/filter/find/reduce/some/every + mutation differences

- objects + destructuring + spread/rest + shallow copy
- ES6+ optional chaining/nullish/modules
- this + call/apply/bind + prototype/class basics
- DOM + events + bubbling/delegation
- localStorage/sessionStorage/cookie concept
- Promise + async/await + event loop
- fetch + response.ok + HTTP CRUD
- 20 logical problems + 5 real-world scenarios

END - JavaScript Frontend Interview Guide

Read less, code more, explain clearly.